Media Insight Engine — How the MCP Server Is Built

A walkthrough you can read out loud to explain the MCP server to a teammate.

---

1. The one-sentence version

> *The Media Insight Engine (MIE) is a FastAPI app that turns admissions videos, podcasts, web pages and PDFs into a cited, per-college knowledge base. The **MCP server** is a thin layer, mounted inside that same app, that re-exposes the engine's abilities as ~17 tools an AI client (Claude Desktop / Claude Code / Copilot) can call — so the assistant can both **query** the knowledge base and **manage** it.*

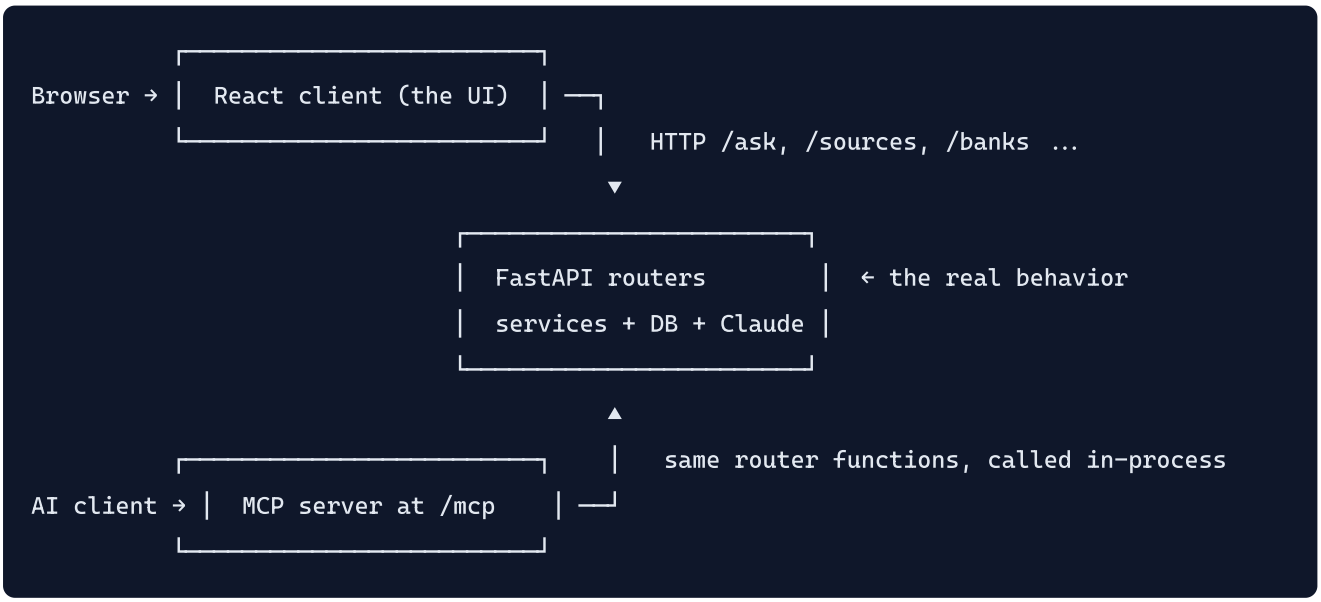
The key design idea: **the MCP server doesn't reimplement anything.** It calls the exact same functions the normal HTTP API already uses, so there is one source of truth for behavior.

---

2. What is MCP, in this context?

MCP (Model Context Protocol) is a standard way to hand an AI model a set of **tools** it can call. Instead of a human clicking buttons in the web UI, the AI client connects to the MCP server, sees a list of tools (each with a name, description, and typed arguments), and calls them to get work done.

So MIE ends up with **two front doors into the same engine**:



3. Where it lives and how it's wired in

Three files matter:

File	Role
`server/app/mcp_server.py`	Defines the MCP server and all its tools
`server/app/main.py`	Mounts the MCP server into the FastAPI app
`.vscode/mcp.json`	Tells the client (VS Code / Copilot) where to connect

Mounting (in `main.py`)

The MCP server is **not a separate process**. It's built once and mounted as a sub-app:

```
from app.mcp_server import mcp, mcp_app
...
app.mount("/mcp", mcp_app)   # streamable-HTTP endpoint at /mcp
```

So the same `uvicorn` process serves `/health`, `/docs`, the `/ask` API **and** `/mcp`. One process, one database connection pool, one deployment.

The lifespan gotcha

A streamable-HTTP MCP server needs a background "session manager" running. FastAPI does **not** run a mounted sub-app's lifespan automatically, so `main.py` starts it by hand:

```
async with mcp.session_manager.run():
    yield
```

If you forget this, the mount exists but `/mcp` never actually works. This is a classic "it's mounted but dead" bug — worth calling out when you explain it.

The trailing-slash gotcha

The app is built with `streamable_http_path="/"` and mounted at `/mcp`, so the real endpoint is `http://localhost:8003/mcp/` — note the trailing slash. The client config reflects that exactly:

```
{ "servers": { "media-insight-engine": { "type": "http", "url": "http://localhost:8003/mcp/"
```

Hitting `/mcp` without the slash is the #1 "why won't it connect" mistake.

4. The one trick that makes it clean: tools call *routers*, not raw services

This is the heart of the design and the thing worth emphasizing.

MIE has layers: **routers** (the HTTP endpoints) sit on top of **services** (the actual logic). The routers add behavior the services don't have on their own:

- `ask` router calls `answers.ask(...)` **and then** `answers.save(...)` — the service alone computes an answer but never persists it.
- `banks` router dedupes questions, validates length (3–2000 chars), and creates the collection row if it's missing.
- `collections` router computes the insights `stale` flag with extra queries.

If the MCP tools called the services directly, they'd quietly skip all of that. So instead, **each MCP tool calls the matching router function in-process.** It gets the exact same behavior, validation, and error handling as the web UI — for free.

Two tiny helpers make every tool a one-liner:

```
def _guard(fn, *args, **kwargs):
    """Run a router function; turn its HTTPException into an MCP ToolError."""
    try:
        return fn(*args, **kwargs)
    except HTTPException as e:
        raise ToolError(f"{e.status_code}: {e.detail}") from e

def _dump(model, value):
    """Coerce the return through the same Pydantic response model → JSON-safe output."""
    ...
```

So a real tool looks like this:

```
@mcp.tool(description="Answer a question using ONLY a collection's sources, with citations.
def ask_collection(collection_id, question, history=None, filters=None) -> dict:
    body = AskRequest(question=question, history=history or [], filters=filters or AskFilte
    return _dump(AskResponse, _guard(ask_router.ask, body=body, collection_id=collection_id
```

Build the request model → call the router through `_guard` → dump the response through `_dump`. Every tool follows that same three-step shape.

Why this matters when you explain it: validation, persistence, the `stale` flag, error messages — none of it is duplicated. Fix a bug in a router and both the UI and the MCP tools get the fix.

---

5. Two more details worth knowing

Sync handlers, run on a thread. The tools are plain `def` (not `async def`). The MCP SDK (v2.1.1) runs sync tool handlers on a worker thread (`anyio.to_thread.run_sync`), so the slow, blocking work — database calls and Claude LLM calls — never freezes the shared event loop. `/health` stays responsive while a long Q&A runs. No manual thread-juggling needed; just define the tool as `def`.

Errors translate cleanly. A router raising `HTTPException(404, "Not found")` becomes an MCP `ToolError("404: Not found")`. So 422 (validation), 404 (not found), and 409 (conflicts like "a sync is already running") all surface to the AI as readable tool errors instead of crashes.

> *SDK note: MCP Python SDK v2 renamed `FastMCP` → `MCPServer` (`from mcp.server.mcpserver import MCPServer`). The tools are hand-authored, not auto-generated.*

---

6. The tool surface (~17 tools)

Grouped the same way the code is:

Agents (the intelligence) - `ask_collection` — cited answer from one collection's sources; persists the answer. - `get_collection_insights` — the home-page synthesis: themes, contradictions, trends, gaps, coverage, timeline. - `refresh_collection_insights` — re-run that synthesis in the background. - `run_question_bank` — batch-ask every question in a saved bank. - `get_bank_run` — status + per-question answers for a run.

Browse / query - `list_collections`, `list_sources`, `get_source` (segments/chapters/quotes with deep links), `get_sync_log`.

Management (write) - `sync_collection` — pull new/changed sources from the collection's config, archive removed ones. - `create_question_bank`, `list_question_banks`, `delete_question_bank`, `get_bank_report` (Markdown). - `list_notes`, `create_note`, `delete_note` — pinned question/answer pairs.

---

7. Long-running work: "start, then poll"

Some jobs take minutes (ingesting sources, regenerating insights, running a whole question bank). Those tools **return immediately** with a status, and there's a matching tool to poll:

Start tool	Poll tool	Done when
`sync_collection`	`list_sources`	sources reach `ready` / `failed`
`refresh_collection_insights`	`get_collection_insights`	`status` leaves `running`
`run_question_bank`	`get_bank_run`	status is `done` / `failed`

Crucially, the **tool descriptions themselves say this** ("Returns immediately; poll X until ..."). That's how the AI knows to wait and re-check instead of assuming the job finished. The server-level `INSTRUCTIONS` string repeats the pattern too, so the model gets the mental model up front.

---

8. How to demo it

1. Start the stack (`make up` / `docker compose up`), server on **port 8003**.
2. Point an MCP client at `http://localhost:8003/mcp/` (see `.vscode/mcp.json`).
3. Ask the assistant something like *"List the collections, then ask the Yale collection what it says about financial aid."* — it will call `list_collections` then `ask_collection` and come back with a cited answer.

Scope / security: **localhost only, no auth** today — it matches the engine's existing localhost CORS posture. Remote exposure and authentication are explicitly out of scope for now.

---

9. The 30-second summary to say out loud

> *"The MCP server is a thin adapter mounted inside the FastAPI app. Each of its ~17 tools just calls the same router function the web UI calls, wrapped in two helpers — one that turns HTTP errors into MCP errors, one that serializes the response. So there's zero duplicated logic: validation, persistence, and the stale-flag all come for free. Tools are sync `def`s so the SDK runs them on a worker thread and slow Claude/DB calls don't block the event loop. Long jobs return immediately and you poll a matching tool. It's localhost, no auth, served at `/mcp/` on port 8003 — mind the trailing slash and the hand-started session manager in the lifespan."*

Source files: `server/app/mcp_server.py`, `server/app/main.py`, `.vscode/mcp.json`. *Full design rationale:* `docs/superpowers/specs/2026-10-16-mie-mcp-server-design.md`.